

❑ Operation: Break the App

Application Security Hackathon Challenge — Talenciaglobal

Duration: 2.5 Hours | Format: Individual | Level: Beginner–Intermediate

Rules of Engagement

- **Your laptop only** — no cloud accounts, no paid tools required
 - **Individual work** — no collaboration during the challenge window
 - **Evidence required** — every exploit must be proven with a screenshot; every fix must be verified to work
 - **One submission** — a single folder containing your fixed code files + a REPORT .md (or PDF)
 - **Hard stop at 2.5 hours** — partial submissions are scored; don't skip challenges, attempt every one
 - **Only listed tools per challenge** may be used for that task
-

Time Allocation (Suggested)

Window	Task
0:00 – 0:15	Read all challenges, set up your environment
0:15 – 0:55	Challenge 1 — SQL Injection
0:55 – 1:30	Challenge 2 — Cross-Site Scripting (XSS)
1:30 – 2:00	Challenge 3 — IDOR & Broken Access Control
2:00 – 2:30	Challenges 4 & 5 — Broken Auth + Security Headers

Pro tip: Read all five challenges before you start. The Flask app you write in Challenge 1 is the base you build on in Challenges 2 and 3. Build it once, extend it.

Tools Required (All Free, Run Locally)

```
# One command to install everything needed
pip install flask

# That's it. All challenge apps are self-contained Flask scripts.
# Additional tools per challenge:
#   Browser DevTools (F12) – built into Chrome or Firefox
#   Burp Suite Community Edition – free download from portswigger.net
#   curl – pre-installed on macOS/Linux; comes with Git for Windows
```

No Docker. No cloud accounts. No paid subscriptions. Every challenge runs with `python app.py` and opens at `http://localhost:5000`.

Challenge 1 — The Broken Login

🕒 35 minutes | Tools: Python, Flask, Browser

Scenario

You are a security intern reviewing the authentication module of a startup's internal tool. The developer built it quickly and never thought about security. Your job: find how to bypass it, then fix it properly.

The Vulnerable App

Save the code below as `app.py` in a `challenge1/` folder and run it with `python app.py`.

```
# challenge1/app.py
from flask import Flask, request, render_template_string
import sqlite3

app = Flask(__name__)

def init_db():
    conn = sqlite3.connect('users.db')
    conn.execute('''CREATE TABLE IF NOT EXISTS users
                    (id INTEGER PRIMARY KEY, username TEXT, password TEXT)''')
    conn.execute("INSERT OR IGNORE INTO users VALUES (1, 'alice', 'password123')")
    conn.execute("INSERT OR IGNORE INTO users VALUES (2, 'bob', 'letmein')")
    conn.commit()
    conn.close()

LOGIN_PAGE = '''<form method="POST">
  <input name="username" placeholder="Username"><br>
  <input name="password" type="password" placeholder="Password"><br>
  <button type="submit">Login</button>
</form>{{ msg }}'''

@app.route('/', methods=['GET', 'POST'])
def login():
    msg = ''
    if request.method == 'POST':
        u = request.form['username']
        p = request.form['password']
        conn = sqlite3.connect('users.db')
        query = f"SELECT * FROM users WHERE username='{u}' AND password='{p}'"
        # ← dangerous
        row = conn.execute(query).fetchone()
        conn.close()
        msg = f'<p>Welcome, {row[1]}!</p>' if row else '<p>Invalid
credentials.</p>'
        return render_template_string(LOGIN_PAGE, msg=msg)

if __name__ == '__main__':
    init_db()
    app.run(debug=True)
```

Your Tasks

Task A — Exploit (8 pts)

1. Log in as `alice` without knowing her password using a SQL injection payload in the username field.
2. Log in as **any user** using only the password field — without knowing any username.
3. Take a screenshot of each successful bypass showing the welcome message.
4. In `REPORT.md`, write the exact payloads you used and explain step by step why each one works.

Task B — Fix (8 pts)

Replace the vulnerable query with a **parameterised query** using `sqlite3`'s `?` placeholder syntax. Your fix must not change any other behaviour — valid credentials must still log in correctly. Paste the corrected code block in `REPORT.md`.

Task C — Explain (4 pts)

In 4–6 sentences, explain *why* string concatenation causes SQL injection and *why* parameterised queries prevent it. Write this as if explaining to a developer colleague who has never heard of SQLi.

Deliverables

- ☐ `app_vulnerable.py` — original vulnerable version (keep for reference)
 - ☐ `app.py` — fixed version
 - ☐ Screenshots of both exploit attempts
 - ☐ Explanation in `REPORT.md`
-

Challenge 2 — The Comment Box That Bites Back

 35 minutes | Tools: Python, Flask, Browser DevTools

Scenario

The startup's blog has a comment section. Nobody sanitised the output. A victim is browsing the page with a session cookie set. Your job: inject scripts, demonstrate what can be stolen, then lock it down.

The Vulnerable App

Save as `challenge2/app.py`.

```
# challenge2/app.py
from flask import Flask, request, render_template_string, make_response

app = Flask(__name__)
comments = []
```

```

PAGE = '''
<h2>Comments</h2>
{% for c in comments %}
  <div>{{ c | safe }}</div>
{% endfor %}
<form method="POST">
  <textarea name="comment" placeholder="Leave a comment..."></textarea><br>
  <button type="submit">Post</button>
</form>'''

@app.route('/', methods=['GET', 'POST'])
def index():
    if request.method == 'POST':
        comments.append(request.form['comment']) # stored directly – no
sanitisation!
        resp = make_response(render_template_string(PAGE, comments=comments))
        resp.set_cookie('session', 'SUPERSECRET42') # no HttpOnly flag!
        return resp

if __name__ == '__main__':
    app.run(debug=True)

```

Your Tasks

Task A — Exploit (8 pts)

1. Post a comment that executes JavaScript and displays an `alert()` box.
2. Post a second comment that reads the session cookie using `document.cookie` and displays its value in an alert — proving what an attacker could steal.
3. Screenshot both alerts. Write both payloads in `REPORT.md`.

Task B — Fix (8 pts)

Apply **two independent fixes**:

1. Remove the `| safe` Jinja2 filter so comments are HTML-encoded on output. Verify the XSS payload is no longer executed — it should display as plain text.
2. Add the `HttpOnly` flag to the `set_cookie` call so JavaScript cannot access it. Verify that after the fix, `document.cookie` returns an empty string in the browser console.

Task C — Explain (4 pts)

What is the difference between Reflected XSS and Stored XSS? Which type did you just exploit, and why is it considered more dangerous? Answer in 4–5 sentences.

Deliverables

- ☐ `app_vulnerable.py` — original version
 - ☐ `app.py` — fixed version
 - ☐ Screenshots of both exploits (before fix) and proof both defences work (after fix)
 - ☐ Explanation in `REPORT.md`
-

Challenge 3 — The Invoice Peek

🕒 30 minutes | Tools: Python, Flask, Browser

Scenario

A billing application exposes invoice data via URL. The developer assumed users would only ever click links to *their own* invoices. They assumed wrong.

The Vulnerable App

Save as `challenge3/app.py`. The app is pre-loaded with data: **alice** owns invoices 1 and 2, **bob** owns invoice 3. The session is hardcoded to alice for this exercise.

```
# challenge3/app.py
from flask import Flask, request, session, render_template_string

app = Flask(__name__)
app.secret_key = 'dev'

INVOICES = {
    1: {'owner': 'alice', 'amount': '$1,200', 'item': 'Consulting'},
    2: {'owner': 'alice', 'amount': '$850', 'item': 'Design Work'},
    3: {'owner': 'bob', 'amount': '$4,500', 'item': 'Server Costs'},
}

@app.route('/')
def home():
    session['user'] = 'alice' # simulate logged-in alice
    return 'Logged in as alice. Try /invoice/1 or /invoice/3'

@app.route('/invoice/<int:inv_id>')
def view_invoice(inv_id):
    inv = INVOICES.get(inv_id)
    if not inv:
        return 'Not found', 404
    # BUG: no ownership check here!
    return render_template_string(
        '<h2>Invoice #{< id >}</h2><p>{{ inv.item }}: {{ inv.amount }}</p>',
        id=inv_id, inv=inv)

if __name__ == '__main__':
    app.run(debug=True)
```

Your Tasks

Task A — Exploit (8 pts)

1. While "logged in" as alice, access bob's invoice (invoice 3) by changing the URL. Screenshot the result showing bob's financial data.
2. Try invoice IDs 0, 4, and 999 — document exactly what the application returns for each. What does this reveal about error handling?

Task B — Fix (8 pts)

Add an ownership check inside `view_invoice`. If the invoice's `owner` field does not match `session['user']`, return a **403 Forbidden** response with the body "Access denied." Verify that alice can still view invoices 1 and 2 but receives a 403 for invoice 3.

Task C — Explain (4 pts)

What is the difference between *authentication* and *authorisation*? Using this challenge as an example, explain how an application can correctly authenticate a user but still be vulnerable to IDOR.

Deliverables

- ☐ `app_vulnerable.py` — original version
 - ☐ `app.py` — fixed version
 - ☐ Screenshots of exploit and of the 403 response after fix
 - ☐ Explanation in `REPORT.md`
-

Challenge 4 — Three Locks, All Broken

 30 minutes | Tools: Python, Flask, Browser DevTools, Burp Suite Community

Scenario

The authentication system has three separate flaws — each independently exploitable. Find all three, demonstrate each one, and apply the correct OWASP-recommended fix for each.

The Vulnerable App

Save as `challenge4/app.py`.

```
# challenge4/app.py
from flask import Flask, request, session, redirect
import hashlib

app = Flask(__name__)
app.secret_key = 'abc' # Flaw 1?

USERS = {
    'admin': hashlib.md5(b'admin123').hexdigest(), # Flaw 2?
    'alice': hashlib.md5(b'alicepass').hexdigest(),
}

@app.route('/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        u = request.form['username']
        p = request.form['password']
        hashed = hashlib.md5(p.encode()).hexdigest()
        if USERS.get(u) == hashed:
            session['user'] = u
            return redirect('/dashboard')
        return 'Wrong credentials'
    return '''<form method="POST">
        <input name="username"><input name="password" type="password">
        <button>Login</button></form>'''

@app.route('/dashboard')
def dashboard():
```

```

    if 'user' not in session:
        return redirect('/login')
    return f'Welcome, {session["user"]}! <a href="/logout">Logout</a>'

@app.route('/logout')
def logout():
    session.pop('user', None) # Flaw 3?
    return redirect('/login')

if __name__ == '__main__':
    app.run(debug=True)

```

Your Tasks

Task A — Find & Describe (8 pts)

Identify all three authentication flaws. For each one:

- Name the flaw
- Quote the exact vulnerable line
- Explain the real-world attack it enables

Use browser DevTools to inspect the session cookie (Application tab → Cookies) and Burp Suite to intercept the login request. The comments (# Flaw 1?, # Flaw 2?, # Flaw 3?) mark the areas — your job is to explain *why* each is dangerous.

Hint: Think about weak Flask secret keys, password hashing algorithms, and what happens to session data on the *server* after a user clicks logout.

Task B — Fix All Three (8 pts)

1. Replace `app.secret_key = 'abc'` with a cryptographically secure random value using Python's `secrets` module.
2. Replace `hashlib.md5` with `werkzeug.security.generate_password_hash` (bcrypt-backed). Update the login check to use `check_password_hash`.
3. Replace `session.pop('user', None)` with `session.clear()` to fully invalidate the session on logout.

Show corrected code for all three fixes in `REPORT.md`.

Task C — Explain (4 pts)

Why is MD5 considered broken for password storage? What makes bcrypt or Argon2 a better choice? Answer in 4–5 sentences — focus on the *technical* reason (speed, salting, cost factor), not just "it is more secure."

Deliverables

- ☐ `app_vulnerable.py` — original version
- ☐ `app.py` — fixed version
- ☐ Screenshots showing cookie inspection and Burp Suite request

- ☐ Explanation in `REPORT.md`
-

Challenge 5 — The Headers Are Missing

🕒 30 minutes | Tools: Python, Flask, Browser DevTools, curl

Scenario

A "production-ready" Flask app is about to go live. A quick look at its HTTP responses reveals it is sending none of the browser security headers recommended by OWASP — and debug mode is still on. Audit it and harden it.

The Vulnerable App

Save as `challenge5/app.py`.

```
# challenge5/app.py
from flask import Flask, render_template_string

app = Flask(__name__)
app.config['DEBUG'] = True # shipped to "production"

PAGE = '''<!DOCTYPE html>
<html><head><title>My App</title></head>
<body>
  <h1>Hello, world!</h1>
  <iframe src="https://evil.com"></iframe>
</body></html>'''

@app.route('/')
def index():
    return render_template_string(PAGE)

@app.route('/error')
def error():
    raise Exception("Database config: host=10.0.0.1, db=prod_users,
pass=hunter2")

if __name__ == '__main__':
    app.run()
```

Your Tasks

Task A — Audit (8 pts)

Run the app and perform the following checks. Screenshot every finding:

1. Visit `/error` — what does debug mode expose in the browser? Why is this dangerous in a real deployment?
2. Open DevTools → Network tab → click the `/` request → inspect response headers. List every OWASP-recommended security header that is **absent**.
3. Identify the HTML-level security problem in the page template itself.
4. Run `curl -I http://localhost:5000/` and paste the output in `REPORT.md`.

Task B — Harden It (8 pts)

Apply all of the following changes:

1. Set `app.config['DEBUG'] = False`.
2. Add an `@app.after_request` function that injects these four headers on every response:

```
@app.after_request
def add_security_headers(response):
    response.headers['X-Frame-Options'] = 'DENY'
    response.headers['X-Content-Type-Options'] = 'nosniff'
    response.headers['Content-Security-Policy'] = "default-src 'self'"
    response.headers['Referrer-Policy'] = 'strict-origin-when-cross-origin'
    return response
```

1. Remove the `<iframe>` from the page template.
2. Re-run `curl -I http://localhost:5000/` after your fix and screenshot the headers now present.

Task C — Explain (4 pts)

Pick any **two** of the four headers you added. For each one: describe the specific attack it prevents, and give a real-world example of how an attacker could exploit its absence. Keep it to 3 sentences per header.

Deliverables

- ☐ `app_vulnerable.py` — original version
- ☐ `app.py` — fixed version
- ☐ curl output before and after (screenshots or pasted in `REPORT.md`)
- ☐ Explanation in `REPORT.md`

Scoring

Area	Points
Challenge 1 — SQL Injection	20
Challenge 2 — XSS	20
Challenge 3 — IDOR	20
Challenge 4 — Broken Auth	20
Challenge 5 — Security Headers	20
Total	100

Bonus: Write a single `pipeline.yml` GitHub Actions file that runs Semgrep, Bandit, or Safety on all five `app.py` files in sequence — no account needed, just the YAML. **+10 pts**

How Each Challenge Is Judged

Criterion	What evaluators look for
Evidence	Screenshots match the described exploit; before/after states are clearly shown
Understanding	Written explanations show you know <i>why</i> something is dangerous, not just how to trigger it
Fix quality	The vulnerability is eliminated; legitimate functionality still works correctly
Completeness	All named deliverable files are present and correctly named

Final Submission Structure

Your submission folder should look like this:

```
your-name-appsec-hackathon/  
├── challenge1/  
│   ├── app_vulnerable.py  
│   ├── app.py  
│   └── screenshots/  
├── challenge2/  
│   ├── app_vulnerable.py  
│   ├── app.py  
│   └── screenshots/  
├── challenge3/  
│   ├── app_vulnerable.py  
│   ├── app.py  
│   └── screenshots/  
├── challenge4/  
│   ├── app_vulnerable.py  
│   ├── app.py  
│   └── screenshots/  
├── challenge5/  
│   ├── app_vulnerable.py  
│   ├── app.py  
│   └── screenshots/  
└── REPORT.md
```

Your `REPORT.md` should contain all five Task C explanations, labelled clearly by challenge number.

Important Reminders

⚠ All exploitation must be performed only on the local applications provided in this challenge. Performing attacks on any live system — even accidentally — is a disqualification offence.

Stuck on a tool? Read the error message carefully. Flask's debug output, browser DevTools, and `curl -v` are your best friends. Reading tool output is itself an AppSec skill being assessed here.

Start with the challenge you feel most confident about. Task C explanations can be written while your fix is running — use the time wisely.

Developed by Talenciaglobal — Application Security Module: Beginner to Intermediate